Arquiteturas Superescalares e Paralelismo em Nível de Instrução (ILP)

Davi Luiz Abrantes Lima - UC25103036
Nicollas Oliveira Melo - UC25102950
Thales Alexandre Ferreira de Sousa Martins - UC25101689

1.Introdução

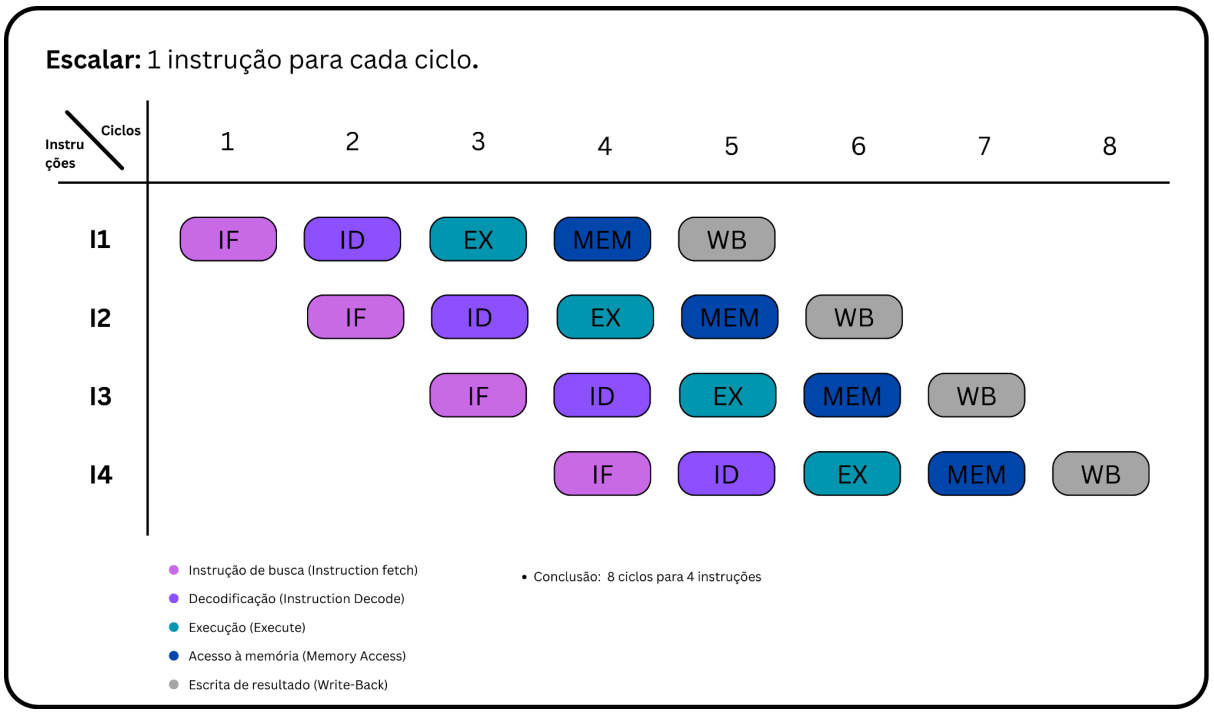
O desempenho de um computador não depende apenas da frequência de operação do processador, mas também de quantas instruções ele consegue concluir por ciclo. Nesse contexto, uma das formas de aumentar essa quantidade é explorar o paralelismo em nível de instrução (ILP), sobrepondo a execução de instruções de um mesmo programa. Dessa forma, as arquiteturas superescalares aproveitam esse paralelismo para executar mais de uma instrução por ciclo, sem que o programador precise controlar diretamente essa execução (Patterson e Hennessy, 2014; Hennessy e Patterson, 2017).

Tendo isso em mente, este trabalho tem como objetivo apresentar os fundamentos do ILP e das arquiteturas superescalares, suas principais técnicas e limitações. Também são apresentados diagramas que ilustram o fluxo de dados e de controle do processador e o desenvolvimento de uma prova de conceito, com a descrição do ambiente de simulação, das ferramentas e dos procedimentos de implementação e teste. A análise dos resultados considera métricas como tempo de CPU, número de ciclos e instruções executadas, discutindo as limitações físicas do processador e seus impactos no desenvolvimento de software de alto desempenho.

2. Fundamentação Teórica e Mapeamento/Diagrama Arquitetural

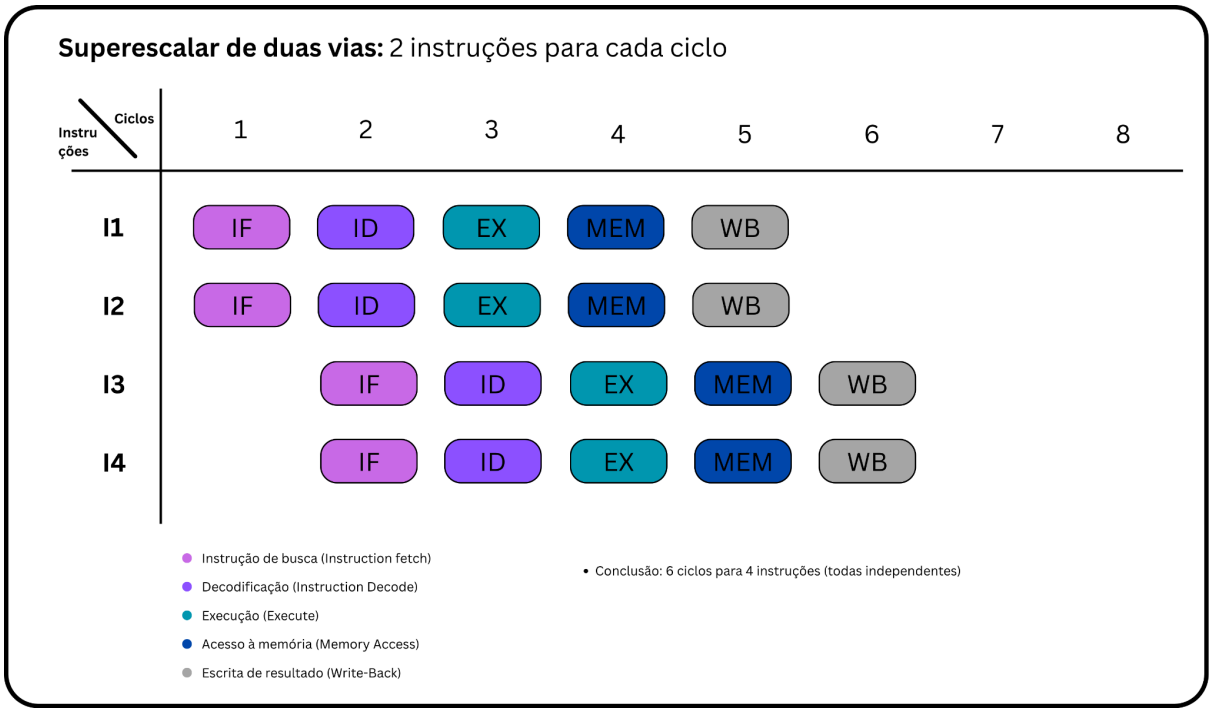
2.1 Pipeline e paralelismo em nível de instrução

Segundo Patterson e Hennessy (2014), o conceito de pipeline divide a execução de uma instrução em estágios, de modo que várias instruções sejam processadas simultaneamente, cada uma em uma etapa diferente, sendo divididas em: busca de instrução (IF); decodificação (ID); execução (EX); acesso à memória (MEM) e escrita de resultados (WB). Em condições ideais, isso permitiria concluir uma instrução por ciclo, tendo o CPI (ciclos por instrução) igual a 1.



(link para melhor visualização)

De acordo com Hennessy e Patterson (2017), o ILP é a medida do quanto as instruções de um programa podem se sobrepor durante a execução, desde que não existam dependências que impeçam essa sobreposição. Consequentemente, quanto mais instruções independentes houver, maior o paralelismo disponível.



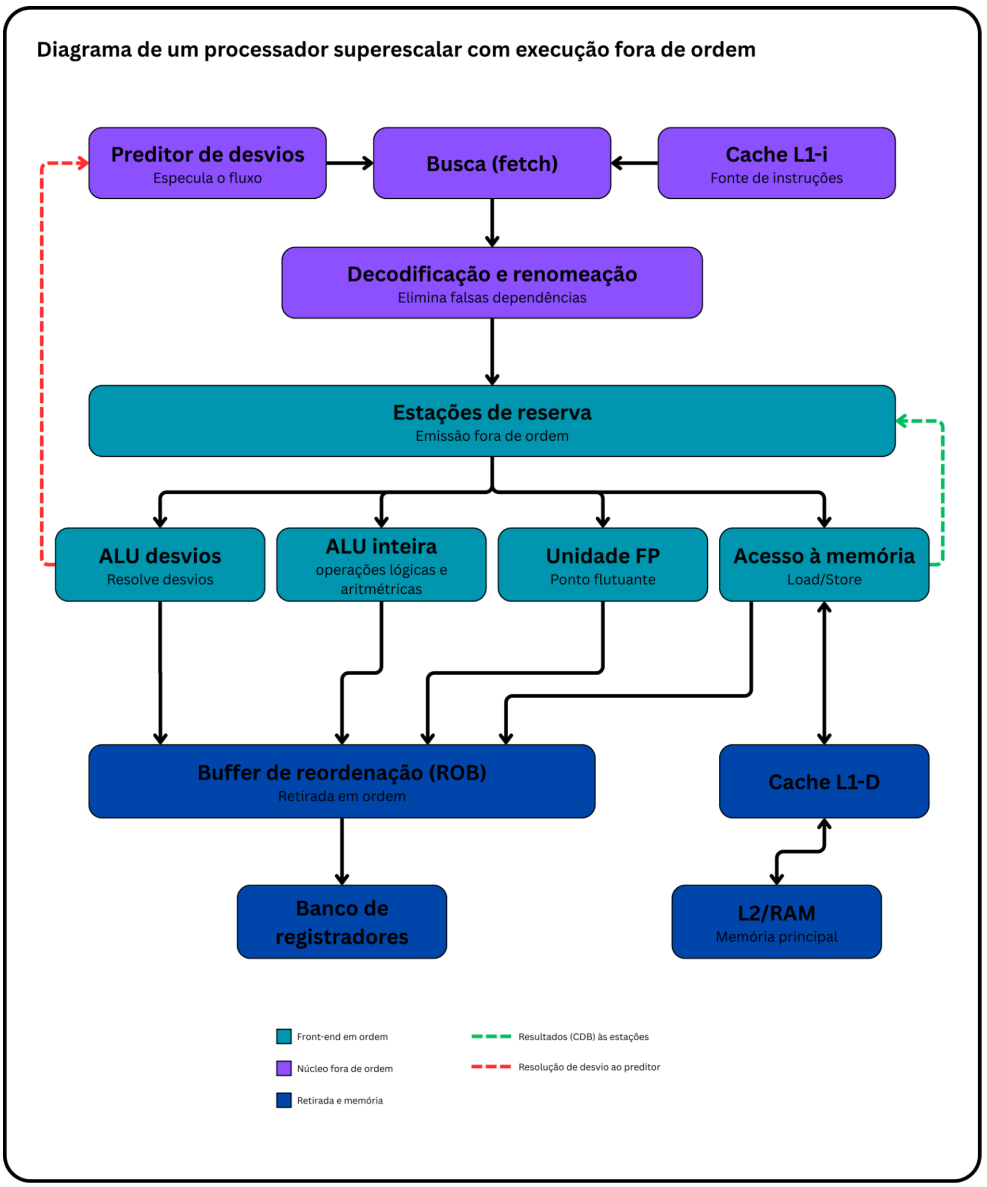
(link para melhor visualização)

2.2 Arquiteturas superescalares

De acordo com Stallings (2017), um processador superescalar emite e executa mais de uma instrução por ciclo, fazendo uso de múltiplas unidades funcionais e de vários pipelines que trabalham paralelamente. Dessa forma, o CPI pode ficar abaixo de 1 e o IPC (instruções por ciclo) pode superar 1. Tanenbaum e Austin (2013) exemplificam essa abordagem com a família Pentium, a qual utilizava dois pipelines paralelos. Dessa forma, as arquiteturas superescalares exploram o paralelismo entre instruções por meio de múltiplas unidades funcionais, permitindo executar mais de uma instrução por ciclo e contribuindo para aumentar o desempenho do processador.

A organização de um processador superescalar é dividido por front-end que opera em ordem, reúne a busca de instruções, a cache de instruções (L1-I), o preditor de desvios, e em seguida o estágio de decodificação e renomeação encaminha as instruções às estações de reserva e consequentemente emitindo para as unidades funcionais fora de ordem logo quando os operandos ficam disponíveis (Shen, 2005).

As unidades funcionais difundem seus resultados no barramento comum de dados (CDB), que os devolvem às estações de reserva e os entrega ao buffer de reordenação (ROB), responsável por retirar as instruções em ordem e atualizar o banco de registradores. A unidade de load/store acessa a cache de dados (L1-D) e, se necessário, os níveis inferiores da memória.

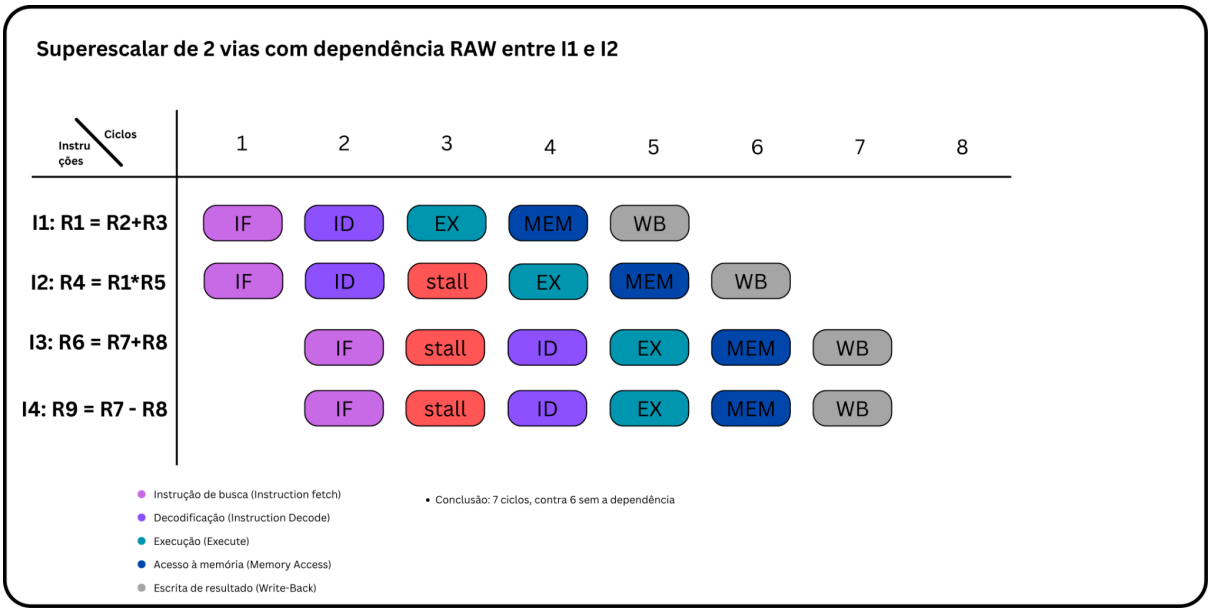


(Link para melhor visualização)

2.3 Dependências e Limites ao paralelismo

Um ponto relevante sobre a arquitetura superescalar e o ILP, é que nem todas as instruções podem ser executadas juntas. Stallings (2017) identifica cinco limitações: dependência verdadeira de dados, dependência procedural (a qual é causada por desvios), conflito de recursos, dependência de saída e antidependência. Enquanto isso, Patterson e Hennessy (2014) descrevem riscos estruturais de dados de controle denominados *hazards*, os quais podem prejudicar a sobreposição de instruções. De tal modo, embora as dependências verdadeiras precisam ser respeitadas, pois uma instrução precisa do resultado produzido por outra, técnicas de hardware e de compilação podem reduzir os impactos dessas restrições sobre o desempenho.

Outro ponto relevante que Patterson e Hennessy (2014) afirmam, é o de que as dependências de dados dividem-se em verdadeiras (RAW, read after write), as quais refletem um fluxo real de valores, e falsas, sendo as de saída (WAW) e as anti dependências (WAR), as quais decorrem apenas da reutilização dos mesmos nomes de registradores. Pelo fato de I2 depender do resultado de I1 (RAW e R1), ela precisa esperar um ciclo e, em um pipeline que executa em ordem, segura também I3 e I4, que são independentes, isso seria um exemplo de uma dependência verdadeira, levando 7 ciclos em vez de 6 ciclos conforme mencionado anteriormente no tópico 2.1. Por conta do escalonamento dinâmico, tal efeito permite que I3 e I4 avancem enquanto I2 aguarda, sem eliminar dependência verdadeira.

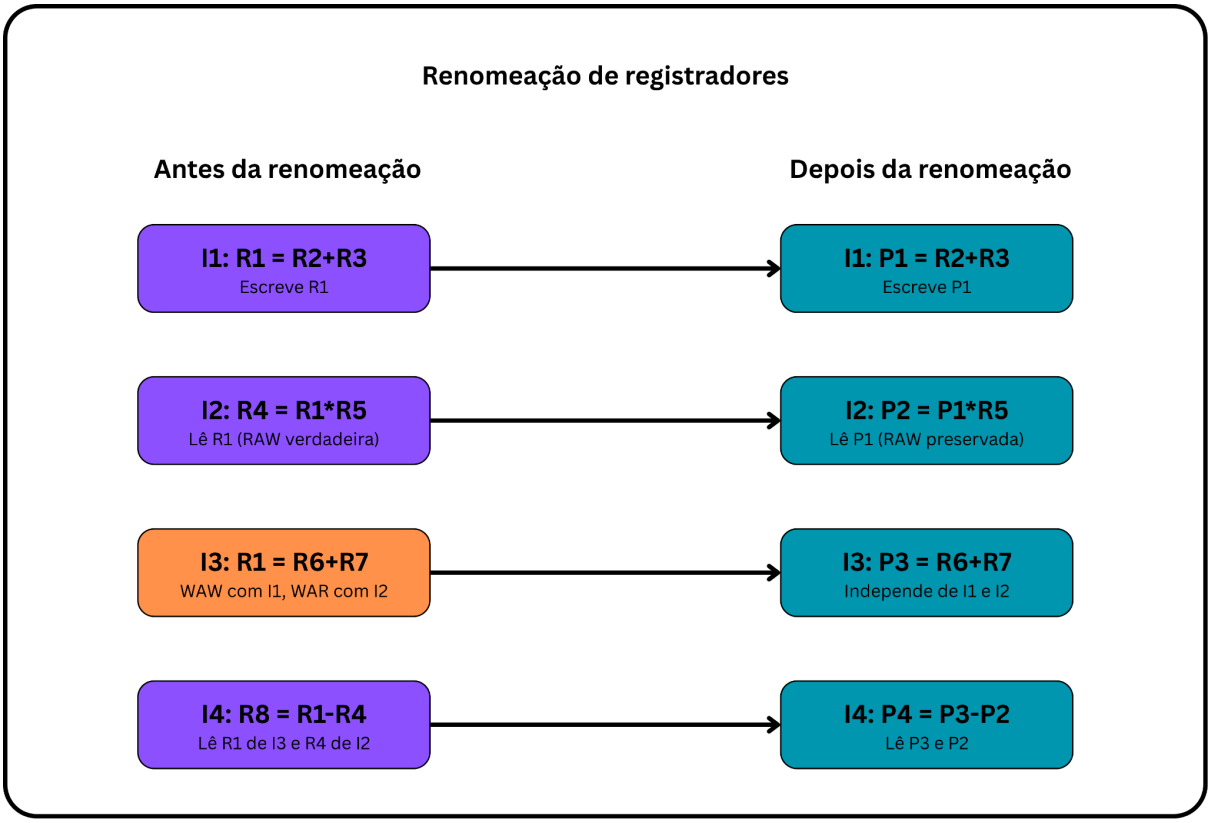


(link para melhor visualização)

2.4 Técnicas para explorar o ILP

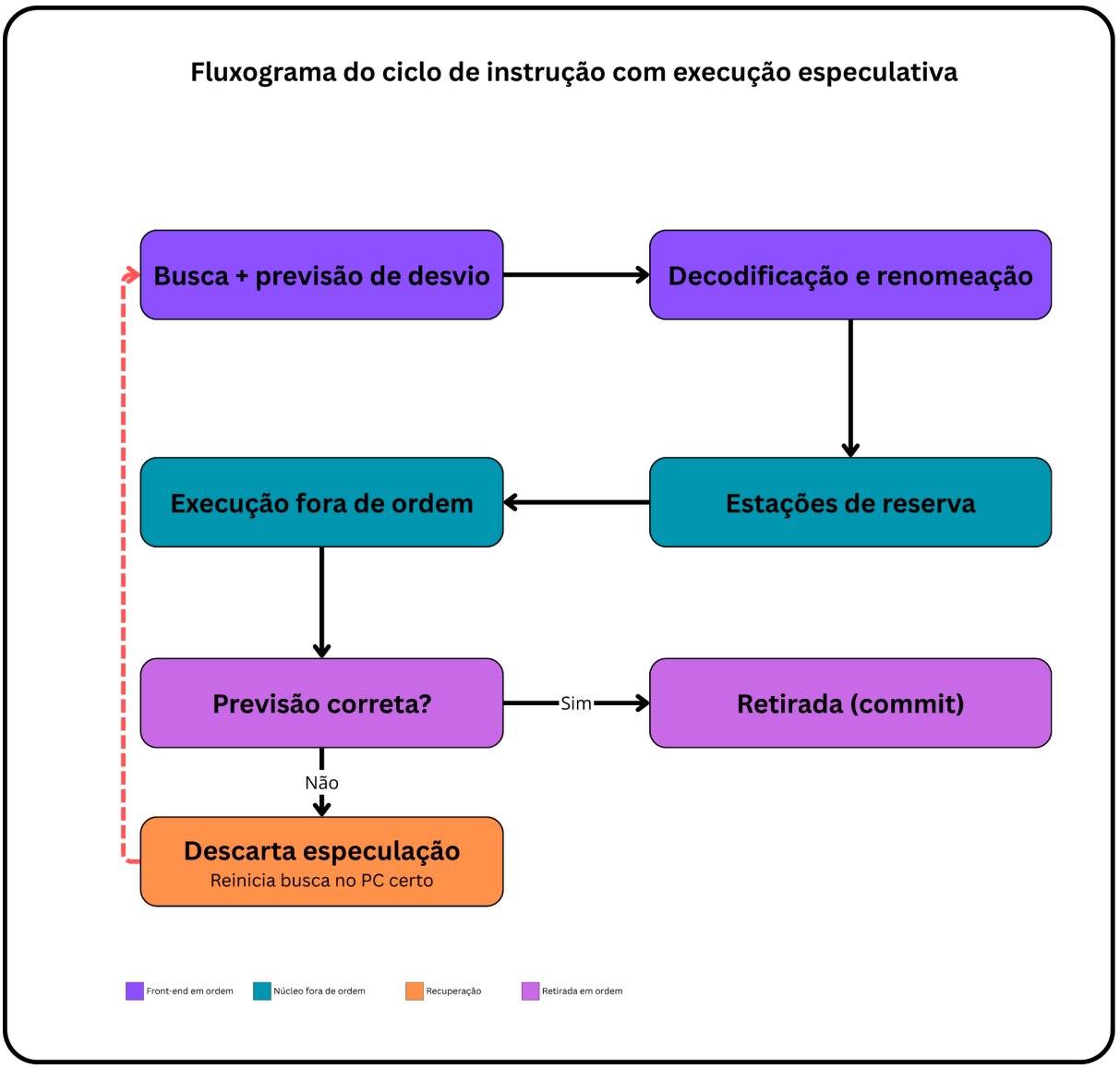
De acordo com Stallings (2017) e Hennessy e Patterson (2017), para extrair mais paralelismo, os processadores superescalares combinam diferentes técnicas:

- **Execução fora de ordem:** instruções prontas podem ser executadas antes de outras que ainda aguardam dados ou recursos. Em projetos com confirmação em ordem, os resultados são confirmados na sequência original do programa. O escalonamento dinâmico que sustenta essa técnica tem origem no algoritmo de Tomasulo (1967), garantindo exceções precisas quando confirmado em ordem, como afirma Smith (1988).
- **Renomeação de registradores:** elimina dependências de saída e anti dependências causadas pela reutilização dos registradores, associando diferentes versões de um mesmo registrador arquitetural e registradores físicos distintos.



(link para melhor visualização)

- **Previsão de desvios e execução especulativa:** o processador prevê o resultado de um desvio e executa instruções antecipadamente. Caso a previsão esteja errada, os resultados dessas instruções são descartados. O ROB impede que uma instrução especulativa altere o estado arquitetural antes de ser confirmada, caso a previsão estiver correta, a instrução é retirada, se estiver errada, o hardware descarta as instruções especulativas, esvazia o ROB e as estações de reserva, reiniciando a busca no endereço correto.



(link para melhor visualização)

2.5 Limitações e o paralelismo em nível de thread

Aumentar a quantidade de instruções que o processador pode emitir por ciclo não garante ganhos proporcionais de desempenho. De acordo com Hennessy e Patterson (2017) isso vai exigir circuitos mais complexos e tende a aumentar a área do chip assim como seu consumo de energia, enquanto dependências e desvios limitam o paralelismo disponível nos programas. Patterson e Hennessy (2014) também ressaltam que as restrições de consumo e de dissipação de calor contribuíram para a adoção de processadores com múltiplos núcleos, os quais permitem executar diferentes threads em paralelo. Apesar disso, o ILP ainda mantém sua relevância para o desempenho de cada núcleo.

Esse assunto surge também quando tem a dependência RAW no tópico 2.3 resulta na redução de ganho mesmo com duas vias, junto com a informação de que cada erro de previsão demonstrado no tópico 2.4 custa o descarte que já tinha sido especulado. Por isso, Stallings (2017) afirma que o ganho adicional passou a vir do paralelismo em nível de thread e de processador

3. Metodologia e Desenvolvimento

3.1 Objetivo da prova de conceito

O trabalho afirma, na fundamentação teórica, que um processador superescalar consegue executar mais de uma instrução por ciclo, que dependências verdadeiras (RAW) limitam esse paralelismo, que a execução fora de ordem e a renomeação de registradores reduzem o impacto das dependências, e que erros de previsão de desvios têm custo. Esta seção testa cada uma dessas afirmações em um processador real.

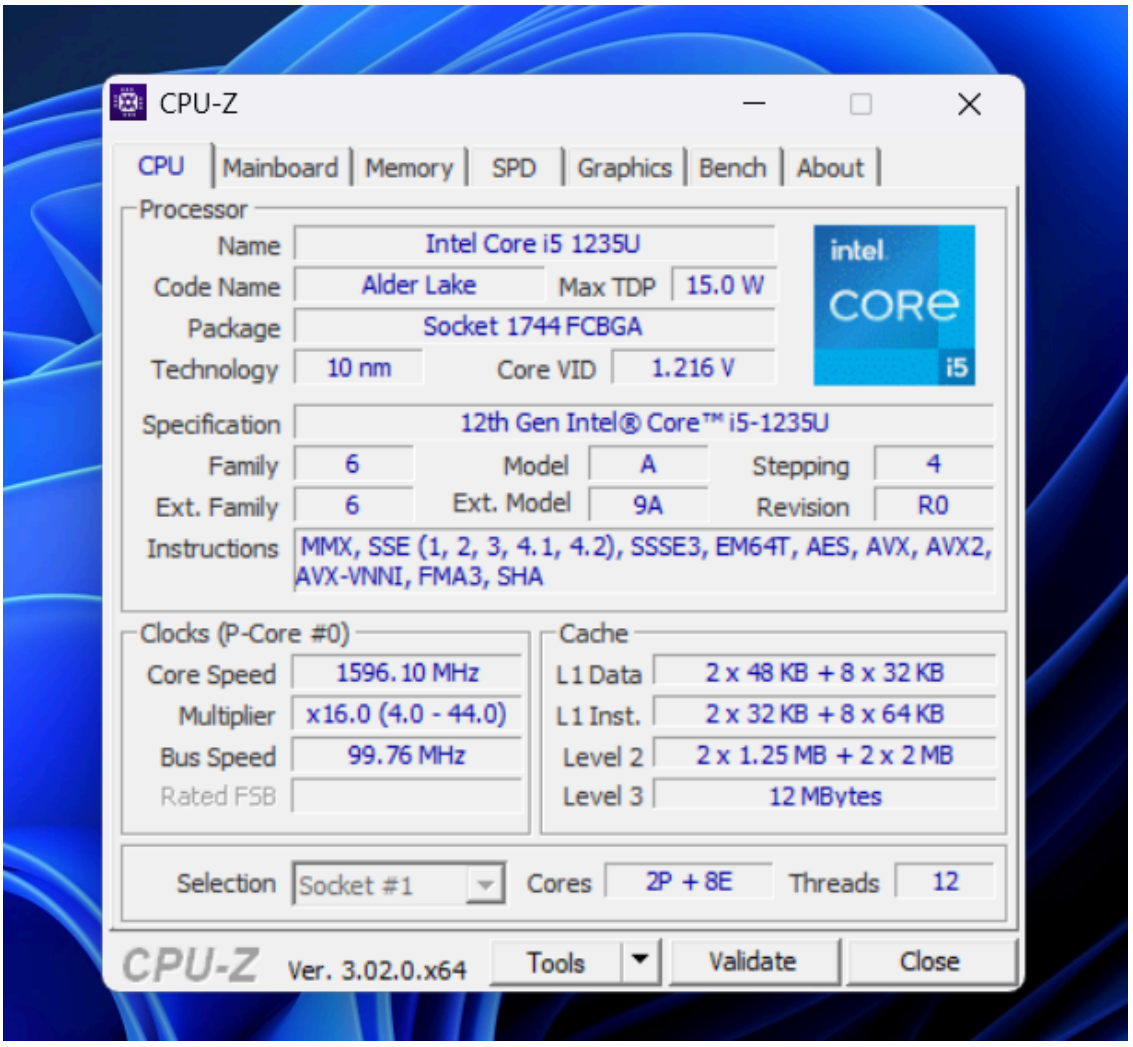
3.2 Ambiente de teste

Tabela 1 – Ambiente *(dados obtidos com PowerShell e CPU-Z; ver Figura 1)*

Item	**Valor**
Processador	Intel Core i5-1235U (Alder Lake, 12ª geração, 10 nm)
Núcleos / threads	2 núcleos de desempenho (P-cores) + 8 de eficiência (E-cores) / 12 threads
Frequência	multiplicador de 4,0× a 44,0× sobre ≈ 99,76 MHz (até ≈ 4,4 GHz no turbo)
Cache L1 de dados	2 × 48 KB (P-cores) + 8 × 32 KB (E-cores)
Cache L1 de instruções	2 × 32 KB (P-cores) + 8 × 64 KB (E-cores)
Cache L2	2 × 1,25 MB (P-cores) + 2 × 2 MB (E-cores)
Cache L3	12 MB compartilhada
Conjunto de instruções	inclui SSE2, AVX, AVX2, FMA3

Sistema operacional	Windows *(preencher com a versão exibida por winver)*
Compilador	MinGW.org GCC 6.3.0 (32 bits)
Desmontador	GNU objdump (binutils) 2.28

Figura 1: captura do CPU-Z (aba CPU)



```
C:\Users\nicol\OneDrive\Desktop\AlgorithmsC\Algorithms>poc.exe 1

[EXP 1] freq_est = 4.29 GHz (1 ADD dependente = 1 ciclo)

cadeias | tempo(s) | ciclos/volta | ADDs/ciclo | IPC_est
   1    |   0.280  |     11.99    |    1.00    |  1.17
   2    |   0.143  |      6.14    |    1.95    |  2.28
   3    |   0.103  |      4.40    |    2.72    |  3.18
   4    |   0.086  |      3.68    |    3.26    |  3.81


C:\Users\nicol\OneDrive\Desktop\AlgorithmsC\Algorithms>poc.exe 2

[EXP 2] freq_est = 4.21 GHz

        variante       | tempo(s) | ns/soma | ciclos/soma
        1 acumulador   |   0.092  |   0.449 |    1.890
        4 acumuladores |   0.024  |   0.118 |    0.496
        8 acumuladores |   0.028  |   0.137 |    0.576


C:\Users\nicol\OneDrive\Desktop\AlgorithmsC\Algorithms>poc.exe 3

[EXP 3] freq_est = 4.30 GHz

independentes | instr/volta | tempo(s) | ciclos/volta | IPC_est
       0      |      3      |   0.140  |      2.01    |  1.49
       4      |      7      |   0.155  |      2.22    |  3.16
       8      |     11      |   0.201  |      2.88    |  3.82


C:\Users\nicol\OneDrive\Desktop\AlgorithmsC\Algorithms>poc.exe 4

[EXP 4] freq_est = 4.32 GHz

nomes de registrador | tempo(s) | ciclos/par | IPC_est
   1 (so eax)        |   0.193  |   1.042    |  2.16
   3 (eax,ebx,edi)   |   0.196  |   1.058    |  2.13
referencia: sem renomeacao, IMUL (3 ciclos) e ADD teriam que ser serializados


C:\Users\nicol\OneDrive\Desktop\AlgorithmsC\Algorithms>poc.exe 5

[EXP 5] freq_est = 4.28 GHz

vetor      | tempo(s) | ns/elemento | ciclos/elemento
desordenado|   0.991  |    3.025    |    12.93
ordenado   |   0.088  |    0.270    |     1.15
razao desordenado/ordenado: 11.2x
penalidade estimada por erro de previsao: 23.6 ciclos (assumindo ~50% de erros)
```

O que cada experimento faz, ilustra e comprova:

Experimento 1: várias unidades funcionais (seção 2.2)

- **O que faz:** executa sempre as mesmas 12 somas inteiras por volta, mas as distribui em 1, 2, 3 e 4 registradores diferentes.
- **O que ilustra:** o núcleo superescalar tem várias ALUs e só as usa em paralelo quando as instruções são independentes. Com 1 registrador, cada soma espera a anterior (RAW). Com 4, são quatro cadeias independentes.
- **O que comprova:** o IPC passa de 1. Ele sobe de 1,17 para 3,81, e o CPI cai de cerca de 0,86 para 0,26.
- **O que mostra além disso:** o ganho não é proporcional. Com 4 cadeias o ideal seria 3 ciclos por volta e o medido foi 3,68. É o retorno decrescente da seção 2.5.

Experimento 2: dependência verdadeira em C (seção 2.3)

- **O que faz:** soma os mesmos 2048 números de ponto flutuante com 1, 4 e 8 acumuladores. O resultado é o mesmo nas três versões.
- **O que ilustra:** o que o programador escreve em C muda o paralelismo que o hardware consegue extrair. É a seção 2.3 aplicada a um programa real.
- **O que comprova:** a dependência RAW custa caro. A versão com 4 acumuladores foi 3,8 vezes mais rápida. Com 1 acumulador, cada soma leva cerca de 1,9 ciclo (a latência do somador). Com 4, cai para cerca de 0,5 ciclo.
- **O limite:** com 8 acumuladores não houve ganho. Isso é consistente com latência de ~2 ciclos × 2 somas por ciclo = ~4 cadeias para saturar.

Experimento 3: execução fora de ordem, o exemplo I1 a I4 (seções 2.3 e 2.4)

- **O que faz:** mantém uma soma dependente em cadeia (o papel de I1 e I2) e coloca ao lado dela 0, 4 e 8 instruções independentes (o papel de I3 e I4).
- **O que ilustra:** em um pipeline em ordem, a espera de I2 seguraria I3 e I4, como no exemplo de 7 ciclos contra 6 do seu texto. Em um núcleo com escalonamento dinâmico, elas passam à frente.
- **O que comprova:** com 4 instruções extras, o total por volta passou de 3 para 7 instruções (+133%) e o tempo subiu só 10%. O IPC foi de 1,49 para 3,16.
- **O limite:** com 8 instruções extras o tempo cresceu 43%, porque elas começam a disputar recursos.

Experimento 4: renomeação de registradores (seção 2.4)

- **O que faz:** executa pares imul + add reusando sempre o mesmo registrador temporário (eax) e, na outra versão, alternando três registradores.
- **O que ilustra:** reusar um nome de registrador cria dependências falsas (WAW e WAR). Sem renomeação, elas serializariam tudo.
- **O que comprova:** as duas versões levaram o mesmo tempo (diferença de 1,5%), com cerca de 1 ciclo por par, bem abaixo dos 3 a 4 ciclos esperados sem renomeação. O hardware dá a cada escrita um registrador físico diferente.
- **Sobre a diferença de 50% da rodada anterior:** ela sumiu ao intercalar as versões e usar o menor de 5 tempos. Isso indica que era variação de medição, e não efeito real.

Experimento 5: previsão de desvios e especulação (seção 2.4)

- **O que faz:** percorre o mesmo vetor, desordenado e depois ordenado, com um desvio condicional por elemento. O código e o número de instruções são idênticos.
- **O que ilustra:** o processador especula o resultado do desvio. Quando erra, descarta o trabalho especulado, esvazia o ROB e as estações de reserva e refaz a busca.
- **O que comprova:** o vetor desordenado foi 11,2 vezes mais lento (12,9 contra 1,15 ciclos por elemento). A penalidade estimada é de cerca de 24 ciclos por erro.
- **Ressalva:** esse valor assume que metade das previsões erra. O número de erros não foi medido.

4. Discussão dos Resultados

Todos os experimentos foram executados em um núcleo de desempenho (P-core) do Intel Core i5-1235U, com o notebook na tomada. Cada medição é o menor tempo entre 5 repetições (3 no Experimento 5). A frequência estimada ficou entre 4,21 e 4,32 GHz em todas as execuções, valor coerente com o limite de 4,4 GHz do turbo, o que dá confiança na calibração. Ciclos e IPC são **estimativas** obtidas a partir do tempo, da frequência estimada e da contagem de instruções do Assembly (seção 4.3).

4.1 Visão geral

Experimento	Afirmação testada	Resultado principal	Resultado
1	O IPC pode superar 1 com várias unidades funcionais (2.2)	IPC estimado de 1,17 a 3,81	Confirmada
2	Dependências RAW limitam o paralelismo (2.3)	4 acumuladores são 3,8× mais rápidos que 1	Confirmada

3	Instruções independentes avançam enquanto uma dependente espera (2.3 e 2.4)	133% mais instruções com apenas 10% a mais de tempo	Confirmada
4	A renomeação elimina WAW e WAR (2.4)	Reusar o registrador não custou tempo (diferença de 1,5%)	Confirmada
5	Erros de previsão descartam trabalho especulado (2.4)	Vetor desordenado 11,2× mais lento	Confirmada

4.2 Experimento 1: várias unidades funcionais trabalhando juntas

Tabela 3 – Resultados do Experimento 1 (12 somas por volta; 14 instruções por volta)

Cadeias	Tempo (s)	Ciclos/volta	ADDs/ciclo	IPC estimado
1	0,280	11,99	1,00	1,17
2	0,143	6,14	1,95	2,28
3	0,103	4,40	2,72	3,18
4	0,086	3,68	3,26	3,81

Com uma única cadeia, cada soma depende da anterior (RAW) e o laço leva 12 ciclos por volta, um por soma. Essa linha é a referência de calibração, então o valor de 11,99 vale por construção. O que importa são as demais linhas. Distribuir as mesmas 12 somas em 2, 3 e 4 registradores independentes reduziu o tempo para 6,14, 4,40 e 3,68 ciclos por volta, e o IPC estimado subiu de 1,17 para 3,81. O CPI, portanto, caiu de cerca de 0,86 para 0,26, bem abaixo de 1. Esse é o comportamento descrito por Stallings (2017) para um processador superescalar, que usa várias unidades funcionais e vários pipelines em paralelo.

O ganho, porém, **não é proporcional**. Com 4 cadeias, o ideal seria 3 ciclos por volta e o medido foi 3,68, com ganho de 3,26× em vez de 4×. A distância do ideal aumenta a cada cadeia adicionada, sinal de que outros limites do núcleo começam a pesar, como o número de unidades disponíveis e a largura de emissão. É a situação que Hennessy e Patterson (2017) descrevem: aumentar a emissão por ciclo não gera ganhos proporcionais.

As três versões calculam o mesmo resultado. Mesmo assim, a de 4 acumuladores foi **3,8 vezes mais rápida** que a de 1. Com um acumulador, cada soma de ponto flutuante espera o resultado da anterior, e o tempo é determinado pela latência do somador: cerca de 1,9 ciclo por soma. Com quatro acumuladores, as quatro cadeias são independentes, o somador recebe trabalho a cada ciclo, e o custo cai para cerca de 0,5 ciclo por soma, isto é, duas somas por ciclo. Isso mostra, em código real, o que Patterson e Hennessy (2014) e Hennessy e Patterson (2017) afirmam: o paralelismo disponível depende de haver instruções independentes, e a dependência verdadeira precisa ser respeitada.

A relação entre os números tem uma leitura simples. O paralelismo necessário para manter o hardware ocupado é aproximadamente a latência da operação multiplicada pela quantidade de operações que o núcleo inicia por ciclo. Com latência próxima de 2 ciclos e duas somas por ciclo, são necessárias cerca de 4 cadeias, que é exatamente o ponto em que o ganho de 3,8× aparece. Passar para 8 acumuladores não ajudou (0,028 s contra 0,024 s). A explicação mais provável é que o hardware já estava saturado. Uma hipótese adicional, não verificada neste trabalho, é a pressão de registradores: o código de 32 bits dispõe de apenas 8 registradores xmm, e a versão com 8 acumuladores usa todos.

4.3 Experimento 2: dependência verdadeira (RAW) em código C

Tabela 4 – Resultados do Experimento 2 (2048 somas por chamada)

Variante	Tempo (s)	ns/soma	Ciclos/soma
1 acumulador	0,092	0,449	1,89
4 acumuladores	0,024	0,118	0,50
8 acumuladores	0,028	0,137	0,58

As três versões calculam o mesmo resultado. Mesmo assim, a de 4 acumuladores foi **3,8 vezes mais rápida** que a de 1. Com um acumulador, cada soma de ponto flutuante espera o resultado da anterior, e o tempo é determinado pela latência do somador: cerca de 1,9 ciclo por soma. Com quatro acumuladores, as quatro cadeias são independentes, o somador recebe trabalho a cada ciclo, e o custo cai para cerca de 0,5 ciclo por soma, isto é, duas somas por ciclo. Isso mostra, em código real, o que

Patterson e Hennessy (2014) e Hennessy e Patterson (2017) afirmam: o paralelismo disponível depende de haver instruções independentes, e a dependência verdadeira precisa ser respeitada.

A relação entre os números tem uma leitura simples. O paralelismo necessário para manter o hardware ocupado é aproximadamente a latência da operação multiplicada pela quantidade de operações que o núcleo inicia por ciclo. Com latência próxima de 2 ciclos e duas somas por ciclo, são necessárias cerca de 4 cadeias, que é exatamente o ponto em que o ganho de 3,8× aparece. Passar para 8 acumuladores não ajudou (0,028 s contra 0,024 s). A explicação mais provável é que o hardware já estava saturado. Uma hipótese adicional, não verificada neste trabalho, é a pressão de registradores: o código de 32 bits dispõe de apenas 8 registradores xmm, e a versão com 8 acumuladores usa todos.

4.4 Experimento 3: execução fora de ordem

Tabela 5 – Resultados do Experimento 3

Instruções independentes	Instr./volta	Tempo (s)	Ciclos/volta	IPC estimado
0	3	0,140	2,01	1,49
4	7	0,155	2,22	3,16
8	11	0,201	2,88	3,82

Cada volta contém uma soma dependente em cadeia (o papel de I1 e I2 na seção 2.3), cuja latência fixa o piso em cerca de 2 ciclos por volta, valor consistente com o do Experimento 2. Ao lado dela, foram colocadas 0, 4 e 8 instruções independentes (o papel de I3 e I4). Acrescentar 4 instruções, que mais do que dobra o total por volta (de 3 para 7), aumentou o tempo em apenas 10%, e o IPC estimado passou de 1,49 para 3,16. As instruções extras foram executadas **enquanto a cadeia dependente esperava**.

Em um pipeline em ordem, a espera de I2 seguraria I3 e I4, como mostra o exemplo da seção 2.3, que leva 7 ciclos em vez de 6. O que se observa aqui é o efeito oposto: o escalonamento dinâmico, cuja origem está no algoritmo de Tomasulo (1967), permite que instruções prontas passem à frente. O ROB garante que os resultados sejam confirmados na ordem do programa (Smith, 1988).

Com 8 instruções extras, o tempo cresceu 43% (2,88 ciclos por volta), pois as instruções passam a disputar a emissão e as unidades funcionais. Esse é o ponto em que o paralelismo disponível deixa de ser "gratuito".

4.5 Experimento 4: renomeação de registradores

Tabela 6 – Resultados do Experimento 4 (8 pares imul + add por volta)

Nomes de registrador	Tempo (s)	Ciclos/par	IPC estimado
1 (só eax)	0,193	1,04	2,16
3 (eax, ebx, edi)	0,196	1,06	2,13

Nas duas versões, cada imul escreve em um temporário sem lê-lo, então não há dependência verdadeira entre as voltas. Na primeira versão, porém, o mesmo registrador (eax) é reutilizado 8 vezes por volta, o que criaria dependências de saída (WAW) e anti dependências (WAR). Sem renomeação, essas dependências obrigariam o processador a serializar os pares, com custo esperado de cerca de 3 a 4 ciclos por par (a latência do imul somada à do add). O medido foi de **cerca de 1 ciclo por par nas duas versões**, com diferença de apenas 1,5% entre elas.

Isso comprova que o hardware associa cada escrita em eax a um registrador físico diferente, eliminando as dependências falsas, como descrevem Stallings (2017) e Patterson e Hennessy (2014). O valor de aproximadamente 1 ciclo por par corresponde ao limite imposto pelas unidades funcionais, e não por dependências. Em uma execução anterior, com uma única medição por versão, a diferença entre as duas havia sido de 50%. Ela desapareceu ao se intercalar as versões e adotar o menor de 5 tempos, o que indica que se tratava de variação de medição (frequência e temperatura) e não de efeito real.

4.6 Experimento 5: previsão de desvios e execução especulativa

Tabela 7 – Resultados do Experimento 5 (32768 elementos × 10000 repetições)

Vetor	Tempo (s)	ns/elemento	Ciclos/elemento

Desordenado	0,991	3,025	12,93
Ordenado	0,088	0,270	1,15

O código e a quantidade de instruções são idênticos nas duas execuções. A única diferença é a previsibilidade do desvio condicional. Com o vetor ordenado, o desvio se comporta de forma regular e o preditor acerta quase sempre: o laço roda a cerca de 1,15 ciclo por elemento. Com o vetor desordenado, o resultado do desvio é aleatório, o preditor erra com frequência, e cada erro obriga o hardware a descartar as instruções especuladas, esvaziar o ROB e as estações de reserva e reiniciar a busca, como descreve a seção 2.4. O resultado foi **11,2 vezes mais lento**.

Assumindo que o preditor erre em cerca de metade dos elementos (dados aleatórios com 50% de chance para cada lado), a penalidade estimada é de aproximadamente **24 ciclos por erro**. O número de erros não foi medido, então o valor deve ser lido como ordem de grandeza. Ainda assim, ele mostra que um único desvio mal previsto custa o equivalente a dezenas de instruções que poderiam ter sido executadas.

4.7 Limitações físicas do paralelismo

Os experimentos confirmam, de forma medida, o que a seção 2.5 afirma. O ganho de paralelismo é limitado por (a) **dependências nos dados**, que impõem a latência (Experimento 2, com 1 acumulador); (b) **dependências de controle**, que tornam o custo dos desvios imprevisíveis para o hardware (Experimento 5); e (c) **recursos finitos do núcleo**, que fazem o retorno diminuir: 3,26× com 4 cadeias em vez de 4× (Experimento 1), nenhum ganho de 4 para 8 acumuladores (Experimento 2) e aumento do tempo quando se acumulam instruções independentes (Experimento 3). Ampliar a largura de emissão exige circuitos mais complexos e maior consumo de energia, e é esse limite físico que, segundo Patterson e Hennessy (2014), levou à adoção de múltiplos núcleos. Por isso, como afirma Stallings (2017), o ganho adicional passou a vir do paralelismo em nível de thread e de processador.

4.8 Impactos no desenvolvimento de software de alto desempenho

- **O código determina o paralelismo que o hardware consegue extrair.** No Experimento 2, a única mudança foi dividir a soma em acumuladores independentes, e o ganho foi de 3,8×. O compilador, por padrão, não faz essa reorganização em ponto flutuante, porque a soma não é associativa e o resultado poderia mudar. Cabe ao programador, quando o resultado final aceita isso, quebrar as cadeias de dependência.

- **Dados e desvios previsíveis importam.** O Experimento 5 mostra que o mesmo algoritmo pode ficar 11 vezes mais lento apenas por causa da ordem dos dados. Ordenar ou agrupar os dados, ou escrever o trecho crítico sem desvios (com movimentos condicionais ou instruções vetoriais), evita essa penalidade.
- **Mais paralelismo não é sempre melhor.** Os retornos diminuem, e o ponto de saturação depende do número de unidades funcionais e de registradores do processador.
- **É preciso medir, e não supor.** Os resultados dependem da microarquitetura, como mostra a observação a seguir.
- **O ILP tem teto por núcleo.** Para ir além, é preciso paralelismo em nível de thread, o que exige que o software seja projetado para isso.

4.9 Limitações da medição

- Ciclos e IPC são estimados, e a frequência é calculada com a premissa de que uma soma registrador+registrador dependente leva 1 ciclo.
- O número de erros de previsão no Experimento 5 foi assumido, não medido.
- Os resultados valem para o P-core usado, uma única máquina e um único compilador (GCC 6.3.0, 32 bits). Em um E-core ou em outro processador, os números seriam diferentes.

5. Conclusão:

Este trabalho investigou como as arquiteturas superescalares exploram o paralelismo em nível de instrução (ILP) e quais fatores limitam esse paralelismo. A fundamentação teórica foi confrontada com uma prova de conceito executada em um processador real, o Intel Core i5-1235U, por meio de cinco experimentos, cada um dedicado a uma das ideias apresentadas na revisão conceitual.

Os resultados confirmaram, de forma medida, as afirmações da literatura. O processador executou mais de uma instrução por ciclo: o IPC estimado subiu de 1,17 para 3,81 quando as mesmas somas foram distribuídas em quatro cadeias independentes, o que reduziu o CPI de cerca de 0,86 para 0,26. A dependência verdadeira (RAW) mostrou-se o principal freio ao paralelismo: reorganizar uma soma de ponto flutuante em quatro acumuladores, sem alterar o resultado, tornou o código 3,8 vezes mais rápido. A execução fora de ordem permitiu que instruções independentes fossem executadas enquanto uma cadeia dependente esperava, de modo que o total de instruções por volta mais que dobrou com apenas 10% de aumento no tempo. A renomeação de registradores eliminou as dependências falsas (WAW e WAR), pois reutilizar o mesmo registrador não custou tempo adicional (diferença de 1,5%). Por fim, o erro de previsão de desvios mostrou seu custo: o mesmo algoritmo foi 11,2 vezes mais lento com dados desordenados, com penalidade estimada de cerca de 24 ciclos por erro.

Os experimentos também evidenciaram as limitações discutidas na seção 2.5. Os ganhos não foram proporcionais: com quatro cadeias, o aumento de velocidade foi de 3,26 vezes e não de 4, e passar de quatro para oito acumuladores não trouxe benefício. Aumentar a capacidade de emissão do processador produz retornos decrescentes, o que reforça a necessidade do paralelismo em nível de thread para continuar ganhando desempenho.

Entre as lições aprendidas, a mais importante é que **o hardware só extrai o paralelismo que o software oferece**. As técnicas do processador (escalonamento dinâmico, renomeação, especulação) são poderosas, mas dependem de instruções independentes e de desvios previsíveis, e essa responsabilidade recai, em boa parte, sobre quem escreve o código. A segunda lição é metodológica: medir exige verificar o que está sendo medido. Na primeira versão da prova de conceito, somas com constante foram resolvidas pelo renomeador do processador e esconderam a dependência que se queria observar, o que produziu uma frequência estimada impossível. A inspeção do Assembly e a checagem de plausibilidade dos números permitiram identificar e corrigir o problema. Isso mostra que resultados de desempenho dependem da microarquitetura e precisam ser interpretados com critério.

Para a carreira em computação e desenvolvimento de software, as implicações são práticas. Conhecer a arquitetura permite escrever código que quebre cadeias de dependência, organize os dados de modo a manter os desvios previsíveis e evite supor que o compilador fará essas otimizações sozinho (o que, em ponto flutuante, ele não faz por padrão). Também ensina a usar ferramentas de medição, como contadores de hardware e inspeção do código gerado, antes de afirmar que uma otimização funciona. E, como o ILP por núcleo tem um teto, projetar software que aproveite múltiplos núcleos e threads passa a ser parte essencial do desenvolvimento de sistemas de alto desempenho.

Como limitações, os resultados foram obtidos em uma única máquina, em um núcleo de desempenho de um processador híbrido, com ciclos e IPC estimados a partir do tempo, uma vez que não havia acesso a contadores de hardware. Como trabalhos futuros, sugere-se repetir os experimentos com contadores de hardware, comparar núcleos de desempenho e de eficiência, e investigar o paralelismo em nível de thread como continuação natural do tema.

6. Referências Bibliográficas:

HENNESSY, John L.; PATTERSON, David A. **Computer architecture: a quantitative approach. 6. ed.** Cambridge: Morgan Kaufmann, 2017.

PATTERSON, David A; HENNESSY, John L. **Computer organization and design: the hardware/software interface. 5. ed.** Waltham: Morgan Kaufmann, 2014.

SHEN, J. P.; LIPASTI, M. H. **Modern processor design: fundamentals of superscalar processors.** Boston: McGraw-Hill, 2005.

SMITH, J. E.; PLESZKUN, A. R. **Implementing precise interrupts in pipelined processors.** IEEE Transactions on Computers, 1988.

STALLINGS, William. **Arquitetura e organização de computadores**. 10. ed. São Paulo: Pearson, 2017.

TANENBAUM, Andrew S.; AUSTIN, Todd. **Organização estruturada de computadores**. 6. ed. São Paulo: Pearson, 2013.

TOMASULO, R. M. **An efficient algorithm for exploiting multiple arithmetic units.** IBM Journal of Research and Development, 1967.